

Item Checker

Project Plan and Step-by-Step Build Guide · SQL (SQLite) application with Tailwind UI · September 2026

1. Project Overview

Item Checker is a simple inventory checking application. It stores the item list in a SQL database, compares **Physical Stock** against **Committed Stock**, and flags items that need attention. Users can import and export the item list as Excel or CSV.

Item	Details
Goal	One place to check item stock levels and spot over-committed items quickly
Fields	Item Name, SKU, Physical Stock QTY, Committed Stock, Active
Database	SQLite (SQL). One table, one view, one trigger
User interface	HTML + Tailwind CSS, opens in Chrome or Edge
Import / Export	CSV and Excel (.xlsx), plus SQLite (.db) backup and restore

2. Technology Stack

Two versions were built. The **standalone version** is the recommended one because it needs no installation.

	A. Standalone (recommended)	B. Server version
File	ItemChecker.html (single file)	item-checker.zip (Flask project)
Database	SQLite running in the browser (sql.js)	SQLite file items.db on the computer
Backend	None, everything runs in the page	Python 3.10+ with Flask
UI	Tailwind CSS	Tailwind CSS
Excel / CSV	SheetJS library	openpyxl + Python csv
How to open	Double-click the file	pip install, then python app.py
Data is saved	In the browser (localStorage) + .db backup	In items.db file
Best for	One user, quick use, no setup	Shared use on a network, future expansion

3. Database Design

3.1 Table: items

Column	Type	Rules
id	INTEGER	Primary key, auto increment
item_name	TEXT	Required
sku	TEXT	Required, UNIQUE, not case-sensitive (HC-001 = hc-001)
physical_stock	INTEGER	Default 0, cannot be negative
committed_stock	INTEGER	Default 0, cannot be negative
active	INTEGER	1 = Active, 0 = Inactive
created_at / updated_at	TEXT	Date and time, updated_at is set automatically by a trigger

3.2 View: v_item_check (status rules)

The view computes **Available Stock = Physical Stock – Committed Stock** and assigns a status. Rules are checked in this order:

Status	Condition	Meaning
OVER-COMMITTED	Committed > Physical	More stock promised than on hand. Check first
NO STOCK	Physical = 0	Nothing on hand
LOW	Available is 5 or less	Running low, consider replenishing
OK	Everything else	Healthy stock level

3.3 SQL schema

```
CREATE TABLE IF NOT EXISTS items (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  item_name   TEXT      NOT NULL,  
  sku         TEXT      NOT NULL UNIQUE COLLATE NOCASE,  
  physical_stock INTEGER NOT NULL DEFAULT 0 CHECK (physical_stock >= 0),  
  committed_stock INTEGER NOT NULL DEFAULT 0 CHECK (committed_stock >= 0),  
  active      INTEGER NOT NULL DEFAULT 1 CHECK (active IN (0,1)),  
  created_at   TEXT      NOT NULL DEFAULT (datetime('now','localtime')),  
  updated_at   TEXT      NOT NULL DEFAULT (datetime('now','localtime'))  
);  
  
CREATE TRIGGER IF NOT EXISTS trg_items_updated AFTER UPDATE ON items  
BEGIN  
  UPDATE items SET updated_at = datetime('now','localtime') WHERE id = OLD.id;  
END;  
  
CREATE VIEW IF NOT EXISTS v_item_check AS  
SELECT id, item_name, sku, physical_stock, committed_stock,  
  (physical_stock - committed_stock) AS available_stock, active,  
  CASE WHEN committed_stock > physical_stock THEN 'OVER-COMMITTED'  
        WHEN physical_stock = 0 THEN 'NO STOCK'  
        WHEN physical_stock - committed_stock <= 5 THEN 'LOW'  
        ELSE 'OK' END AS status,  
  updated_at  
FROM items;
```

4. Features

- **Dashboard cards:** Total Items, Active, Physical QTY, Committed QTY, Over-committed count
- **Item list:** search by Item Name or SKU, filter by Active and Status, color-coded status badges
- **Add / Edit / Delete** items, with a one-click Active toggle
- **Validation:** duplicate SKU is blocked, stock cannot be negative, Item Name and SKU are required
- **Import:** CSV or Excel. Existing SKU is updated, new SKU is added (UPSERT). Bad rows are skipped and reported by row number
- **Export:** Excel (.xlsx), CSV, or SQLite backup (.db). Export format matches the import format, so a file can be exported, edited in Excel, and imported back
- **SQL Console:** run your own SQL queries, with ready-made buttons for common checks

5. Step-by-Step Build Plan

These are the phases followed to build the application, in order.

Step	Phase	What was done	Output
1	Requirements	Listed the fields (Item Name, SKU, Physical, Committed, Active) and the needed functions: list, check, import, export	Feature list
2	Database design	Designed the items table with rules (unique SKU, no negative stock), the status view, and the updated_at trigger	schema.sql
3	Data layer	Wrote the SQL queries for listing, filtering, add, update, delete, and totals	Query functions
4	User interface	Built the page with Tailwind: header, stat cards, filter bar, table, and pop-up forms	Main screen
5	Import	Read CSV/XLSX, matched column headers, validated each row, and saved with INSERT ... ON CONFLICT (UPSERT)	Import function
6	Export	Generated Excel and CSV with the same headers as import, plus a full .db backup	Export function
7	SQL Console	Added a query box and result table for direct SQL checks	SQL tab
8	Testing	Tested add, duplicate SKU, filters, import, export, re-import, delete, and reload	Test results (section 8)
9	Packaging	Packed into one HTML file for easy opening; server version kept as a zip	ItemChecker.html

6. How to Use (Standalone Version)

#	Step	Instructions
1	Open the app	Download ItemChecker.html and double-click it. It opens in Chrome or Edge. Internet is needed the first time to load the libraries.
2	Clear the sample data	The app starts with 5 sample items. Click Clear sample data in the yellow banner to start fresh.
3	Prepare your file	Click Import then Download import template . Fill it in Excel using the 5 columns.
4	Import	Click Import , choose your .csv or .xlsx file, then click Import . A message shows how many were added, updated, and skipped.
5	Check the stock	Use the Status filter and pick OVER-COMMITTED or LOW to see items that need action.
6	Update items	Click Edit on a row to change quantities, or use the toggle to mark an item inactive.
7	Export	Click Export and choose Excel or CSV for reports, or SQLite backup (.db) to keep a copy of the database.
8	Back up regularly	Data is saved in the browser. Export a .db backup regularly. To restore, use Import and choose the .db file.

7. Import File Format

Item Name	SKU	Physical Stock QTY	Committed Stock	Active
Nordic Dining Chair	HC-CHR-001	120	30	TRUE
Oak Coffee Table	HC-TBL-014	15	18	TRUE
Rattan Floor Lamp	HC-LMP-077	0	0	FALSE

- Item Name and SKU are required. Other columns default to 0 (stock) and TRUE (Active) if blank.
- Active accepts TRUE/FALSE, YES/NO, 1/0, or Active.
- Numbers with commas (1,200) are accepted.

8. Test Results

Test	Expected	Result
Load app with sample data	5 items shown	Pass
Add item with an existing SKU (different letter case)	Blocked with message	Pass
Add new item	Item count increases	Pass
Filter OVER-COMMITTED	Only HC-TBL-014 (15 vs 18) shown	Pass
Export Excel / SQLite backup	Files download correctly	Pass
Import CSV (5 rows)	5 new, 0 updated, 0 skipped	Pass
Re-import exported Excel	0 new, 5 updated	Pass
SQL Console summary query	Correct totals per status	Pass
Reload the page	Data is still there	Pass
Delete item	Removed after confirmation	Pass

9. Limitations and Notes

- The standalone version saves data **in the browser**. Clearing browser data or using another browser or computer will not show the same data. Use the .db backup to move or keep data.
- It is designed for **one user**. For several users sharing the same data, use the server version on a shared computer.
- Internet is needed to load Tailwind, sql.js, and SheetJS when the page opens.

10. Possible Next Steps

- Login with user roles (viewer, editor, admin)
- History log of every stock change (who, when, old and new values)
- Warehouse or location column for multi-warehouse checking
- Sync with Zoho Inventory to pull Physical and Committed stock automatically
- Adjustable LOW threshold instead of the fixed value of 5